Software Architecture

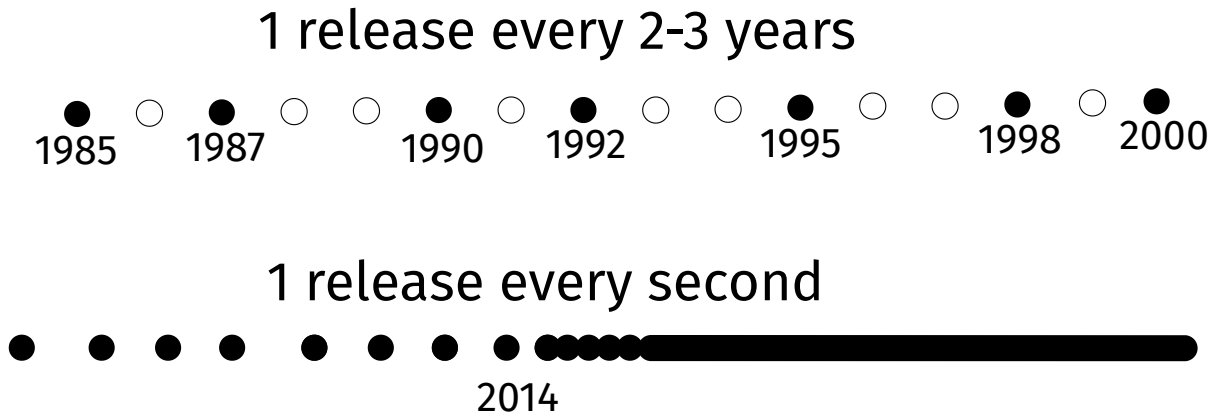
Deployability, Portability and Containers

9

Contents

- Deployability Metrics
- To Change or not to Change
- Continuous Integration, Delivery and Deployment
- Platform Independence: Virtualization and Containers

The Age of Continuity



After we have discussed the qualities of a software architecture at design time, today we're ready to make the big transition: we're going to deploy our system in production. This is the lecture in which we focus on how easy it is to deploy the software system in production that we are designing and what kind of architectural decisions we can make that affect this deployability. Before we can deploy, we have to build it and make sure that it can be executed in a container in a runtime environment. So today we will also discuss virtualization and containers in the second part with an eye towards portability as well. From the next lecture on, we will focus on runtime qualities such as scalability, availability, flexibility.

Why is deployability important? The way that we built our software systems has been undergoing a dramatic change as we have been switching away from an age in which software was built on a slow moving cycle with a major release of a software system every couple of years. It's important to worry about this quality and to design the proper system to build our system because we have been changing the frequency with which we ship releases and deploy software. In the past, making a release was a major undertaking that happened relatively unfrequently, so here you can see for example the first decade of Windows releases happening every two or three years. Somehow between that time and today we have dramatically increased the frequency with which software is released. Some people actually speak about the end of the software release: from the user perspective there is no more the concept of what is a version of software system, because every time you access a system, it actually has changed. Today deployability is about making releases continuously, so that we can improve continuously the qualities of our software: add more features, fix more bugs, and do so with confidence at high speed.

Some major cloud providers have announced, for example, that in 2011 they would make new releases every 10 seconds. Then in 2014 they actually increased the speed tenfold: now they are doing new releases every second. How can achieve this high speed of release without sacrificing the other qualities? That is the major challenge that has been addressed in the industry and which we will discuss in this lecture.

Deployability Metrics

- Latency:
  - What's the delay between developers writing code and users experiencing the improvement?
  - What's the delay between users giving feedback and developers releasing a fix?
- Throughput:
  - How many releases are shipped to production every hour/day/month?

Deployability is a quality attribute that can be measured. And there are many ways to measure the deployability of a system.

One concerns the latency, which we can define in terms of what is the delay between the code the developer writes and the user observing the effect of that code after it gets deployed in production. If the developer is writing a new feature: how long does it take before this new feature gets delivered to the user? And if the developer is fixing a bug: how long does it take before the user that reported the bug observes that the bug is no longer there?

These are examples going forward, but you can also observe and measure deployability in the backward direction. For example, a user reports a bug, gives some feedback about something that should be addressed. Of course, you have to route the feedback to the right developer and make a decision whether you will actually implement it or not. Afterwards you can measure again how long it takes after the developer has written the code and the user has seen and accepted the improvement.

So how long does it take you to do that? Is it just like a click over the 'deploy' button and then the user automatically gets an update and no additional effort is required to benefit from the improvement? Or is this like a major project over several months to successfully go through this cycle once.

Considering the throughput, we can measure deployability for the whole system in terms of how many releases do you ship to production over time? You do a release every two years, or you do multiple releases every second? That would be the expected range for deployability throughput.

Deployability Metrics

- Time:
  - For how long will the system not be available during upgrades?
  - Is the release late with respect to the marketing target deadline?
- Cost:
  - How many people are involved in a release?
  - How many Virtual Machines are needed to host the software production pipeline?

Let's also consider how expensive it is to do a deployment. Is this something that one developer can do with a click to launch the corresponding script in their IDE? Or does it entail a major undertaking with multiple teams involved, where developers have to coordinate with operations, to schedule a suitable upgrade window? Or do multiple development teams have to synchronize? For example, one team is making a release for the server side and all the affected client endpoints need to be redeployed as well.

When you're doing an upgrade, is the system availability going to be affected? Are the users going to see a little message apologizing for the ongoing maintenance: you cannot use the system while it's been upgraded, take a long coffee break. Your architecture has a better deployability if, during the change, as a new version of the system is deployed, the users do not notice. It is very important not to be disruptive of the productivity of your users. If you have to stop what they're doing to wait for an upgrade, consider upgrading over the weekend. Disruptive, unplanned upgrades in the middle of the work day can be a very expensive proposition.

Often one hears marketing announcements about the availability of new software products. For example, a new game will be released on the Black Friday or the Christmas seasons. It is critical to hit this particular launch window. If you are late and cannot ship it in December, probably this is a big missed opportunity for revenue. We can also measure deployability in terms of whether the release is shipped on time so we can meet our customers expectations generated by the marketing campaign.

Another aspect for measuring the cost of deployability can be observed by assessing how expensive is the infrastructure to do the build and the testing along the software production pipeline. Can you build your system on your developers laptop? or do you need a whole cluster of virtual machines to process the compilation of the code and the parallel execution of all the automated quality assurance tests?

There are some systems with millions of lines of code which require several hours to compile and the complete testing will run for a couple of days. In this situation, making a build is very expensive computationally, and if you make a small change and then you

have to wait for the whole night before you see the release of the result of the nightly build, your productivity becomes limited by the time it takes to get feedback due to the expensive deployability of large architectures.

Deployability Metrics

	Traditional	Continuous
Latency	High	Small
Throughput (Releases/Time)	1/Years	Many/Day
Downtime	Noticeable	0
Deadline	Death March	Not Applicable
People	Dedicated Team of Release Engineers	0 (Full Automation)
Infrastructure	?	Investment Required

Traditional deployments – as an extreme example – have very low throughput of yearly releases; when making a deployment, users will notice. If you need to catch a released deadline, you will attempt to do so through so-called death marches, which will lead to burnout and high developer turnover. Since every release is risky, requires heroic efforts, you do not want to go through crunch times too often. Making releases requires a big investment into a team of release engineers who take the code, build it, test it and package it so that it can be released by copying it on some website so that it can be downloaded and installed. In the good old days this also included people or robots burning the software on CDs and placing them into shrink wrapped boxes. There was no viable concept of automated builds and continuous integration delivery pipelines. Deployment was an ad-hoc, manual, error prone, very slow and expensive.

Nowadays, if you want to make a release, you can do it quickly. You can do it often. Your users do not notice, as you can apply techniques that help you to minimize the disruption involved into making a change to a production system. Since the release is continuous, you don't have a target window anymore, you just do it all the time. There is no longer a single opportunity, you can continuously improve the system. This has become possible because the process is fully automated: you just have to invest into setting up the pipeline with the proper tooling and the proper infrastructure to build your software. Then the pipeline reliably runs unattended: shipping a release has matured from a craft involving black arts into a well oiled industrial workflow.

Release

Opportunity or Risk?

- Deliver new features
- Fulfill requirements
- Bug fixes
- Performance improvements
- Generate revenue
- Retain and grow customers
- Match or exceed competition

- New failure modes
- New bugs
- Higher resource capacity required
- Peak of support calls
- Deployment may fail
- Deployment failure may be unrecoverable

Every time you introduce a change, you need to make a new release. When you make an improvement of your software, it's a big opportunity because you are going to deliver new features, fix bugs and in general attempt to make your users happier because you better fulfill their requirements, so this is a positive opportunity to address users constructive feedback. New releases can be expected to improve not only the correctness but also extra functional qualities, for example: better performance. Another consequence of shipping a new release is the opportunity to make your customers pay for the upgrade. This is one of the business models for software components: every time you make a new release, your users have to pay to install the upgrade. Some customers have a tendency to flock to freshly released software. Others, more conservative, wait for the release to age before daring to install it. From a business point of view, making a release is critical to retain existing customers and get new ones, since your software now delivers more features than the competition, or it has been finally improved to catch up with the competition. These are all reasons why you should not hesitate before making a new release of your software.

However, there is also the flip side in which when you make a new release, all of a sudden the system behaves in a new way - sometimes in an unexpected way which may lead to new failure modes. You already knew in which configuration and for which input your previous release was unstable. You make a change and now you need to discover even more cases in which things can go wrong.

For every bug that you fix, you introduce a couple of new ones; you have to be careful not to catch those bugs before you put them in. The performance might be improved, but maybe also your capacity requirements have increased, so now we need more resources than before, so it has just become more expensive to run the system.

Your new features interfere with the normal operations of the users. They confuse them and lead to an increased cost for training and support, helping users get used to the change: Why did this change? Where did my favorite feature go? Why did you move the menu somewhere that I cannot find it anymore?

While you're in the middle of the deployment, things can go wrong. The deployment itself is a dangerous operation which changes the state of running system into an unknown one. If the new state works, congratulations for your success! But if the deployment fails, what happened to the previous release? Avoid ending up in a situation in which the new release is broken and the old release is gone. Your users will definitely notice. That's why weekends have two days: Saturday to deploy the new release, Sunday just in case, to clean up or revert back to the previous one. Always have a time buffer and a strategy to recover from a broken upgrade. If you're not careful in the way that you do the deployment failures could even be unrecoverable.

Thanks to virtualization this should never happen anymore, but when people used to work with physical computers. You have a physical object in which you installed an operating system. And then you have a disk with the software and the data in a certain version. Since it's expensive to have a new physical server on which to place the new release, some brave system administrators may just decide to install the new version over the existing one. If you do so and something goes wrong, the old one is gone - because you have just formatted the disk and forgot that you were supposed to migrate the old data. While recovering the old software may be possible with enough effort, losing data during deployments is unforgivable.

No Change = No Risk

The only software which does not need to change is dead software which nobody uses

Minimize risk due to change

instead of

minimizing change

Release many small improvements often

Every release should be reversible

For these reasons, experienced operators grow into being conservative. When they hear about your new release, their reflex is to ask: Are you really sure you want to install the update? Because after all there is a risk that it may not work as it used to. It is reasonable to hesitate before going from a known state to an unknown one.

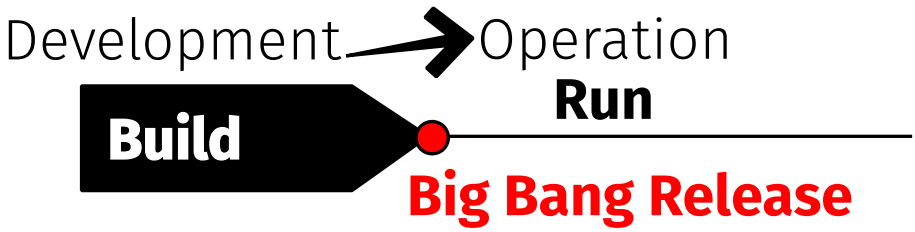
In the extreme case: simply don't make changes, ever. Stay in known territory. If you don't change, there is no risk: problem solved. However, the only software which doesn't need to change is dead software used by nobody. Anyone using your software will always find some ideas about how to improve it or make friendly suggestions to make some corrections.

If you avoid risk by never daring to make changes, your release process takes years before you actually gather the courage to actually make the step. What you want instead is to minimize the risk of change as opposed to reducing changes themselves.

How do you do that? You make the magnitude of the change small, you make those changes often. This way you get used to making the changes and making changes to your system is a normal, frequently executed operational process: not something that you just do once in your lifetime. You release and deploy every day so that it becomes a habit and you learn how to do it properly.

Also, you shouldn't make changes without a safety net. Like database transactions: begin the deployment; apply all the changes; commit. If something goes wrong during a transaction, roll back to the initial state. If you can find a way to make a release of software work like atomic database transactions, which go from a consistent state to another consistent state, then you have the safety net which should reduce the risk of making a failed deployment that will destroy your system, tarnish your organization's reputation and eventually get you fired.

Release Frequency



If it hurts, do it more often (so that practice makes perfect)

In the classical waterfall, a lot of time is spent in development building the software until it is feature complete. All tests are green. Eventually the development project is finished. Developers package and ship the release. And then they throw it over the wall to the operations team. The operations team has never done anything until this point and they take over. They install the system, they run it and after this transition one could literally fire your developers because you don't need them anymore. The software is done, the project has been completed. So developers can move on to another project as the operations department runs the system.

What can go wrong from a development point of view? you think that you're done because you have clicked compile. And you generated some executable artifacts. Given some tests, maybe you even run them and check they pass. Still, even if it passes all the tests, there is no guarantee that is actually correct. The real test will start when actual users start to run it. But that's no longer your job. That is somebody else's problem.

While it's very rare that the system is perfect on the first shot, in the first release. The idea is that if this transition is expensive, complicated, risky – in other words: if it is a pain – then you should do it more often so that it becomes less painful. This situation is called 'Big Bang' release. Big bang releases are a once-in-a-lifetime event (The Big Bang only happened at the beginning of the universe). That's the only chance you have. So if it goes well, you are done. If it goes wrong, there is no second chance.

There are many reasons making challenging to achieve a successful Big Bang release, but this was the traditional model where you have the software built by one part of the organization and then a different group takes over to execute it and will need to deal with whatever operational problems emerge as a consequence of a successfully shipped development project. In this setup, there is very little feedback going back from operations to developers.

How do we make releases happen in a continuous way? How do we do it more often?

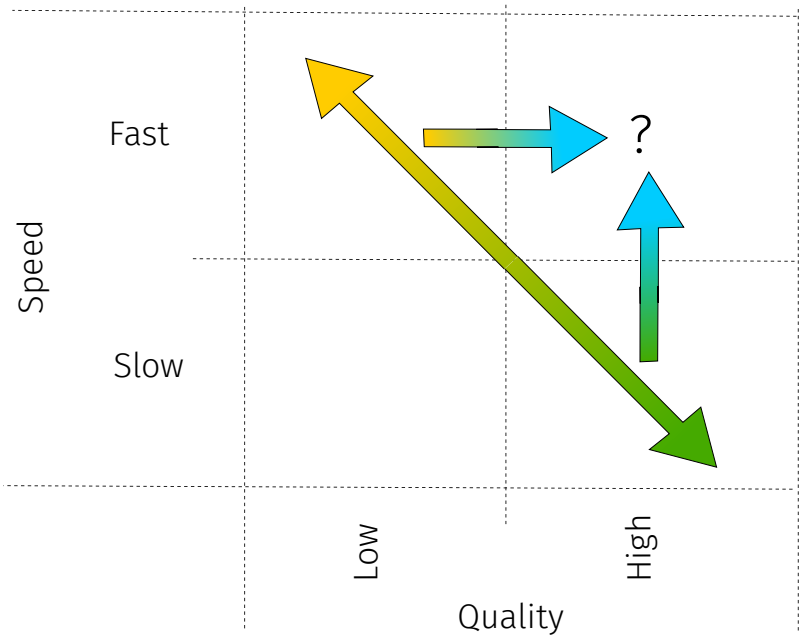
We are going to make the operation and the development happen in parallel, so they're no longer sequential steps. They're no longer: first we develop and then you operate, but while we develop, we also operate the system. The transitions between the system being built and the changes being introduced during development cycles and the deployment and operation happen continuously.

This doesn't mean that continuous deployments will be always successful. You can also have failed builds, broken releases and unsuccessful deployments. Look at the timeline for the red dots: The first releases are good, but then something goes wrong while we keep improving the system and eventually we hit another good release. What is fundamental here is that we have a continuous feedback cycle between developers that make the changes and operations that watch the impact of changes on users and feed their feedback back to the developers for further improving the system.

There is no separation anymore between who is responsible for applying the changes and who is living with the consequences of that. Because of such tight feedback cycle between the two sides and by making the release something that happens continuously, we have a lot of opportunities for trying out new ideas for making improvements that might not necessarily stick. Maybe not all the users appreciate them, so we can always go back and undo those changes.

This is a paradigmatic shift between building first and operating later and doing both at the same time, as well as going from separate teams dedicated to each activity to the same team involved with both activities.

Speed vs. Quality



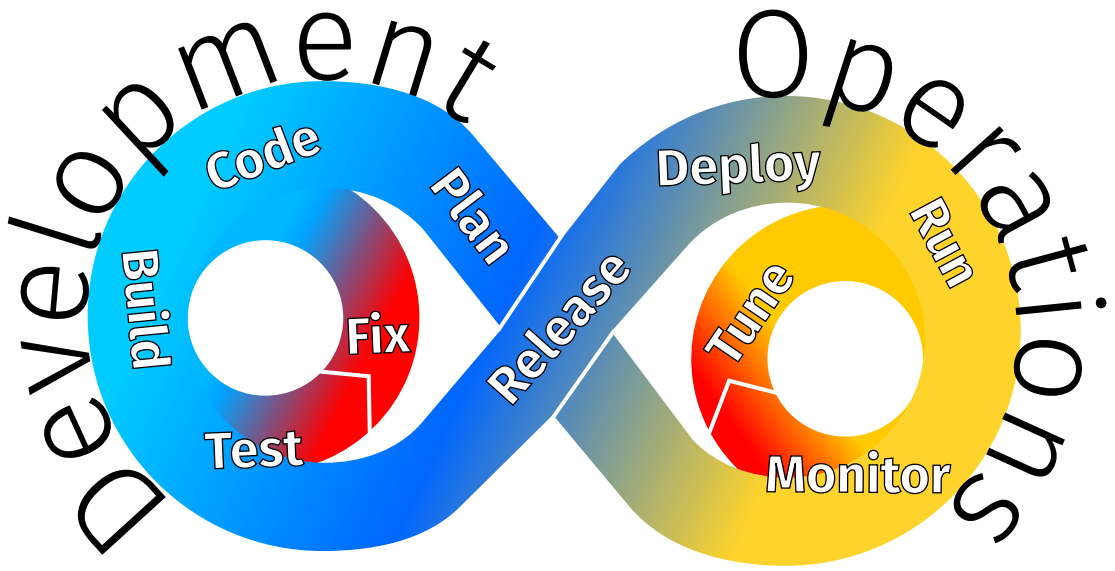
Thanks to Automation, Speed vs. Quality is no longer a Trade-Off

Another change in in perspective that happens with DevOps and continuous releases is that there used to be a trade off between the speed with which you develop and release and the quality of the result.

Conventional wisdom was that you can either have low quality when you go fast. The faster you develop something, the less time you can dedicate of doing so with good quality (both external and internal). Would you like to improve the quality? That's OK, but then you have to slow down.

Doing continuous releases wouldn't work if this trade off was still true, because we cannot sacrifice quality to increase the release throughput. Otherwise we will always be in a broken state with a failed release.

When you start to automate all your quality assurance and testing tasks, when you automate your production process to build and package the images and releases so that the process becomes not only fast but also error free, controllable and repeatable, then you can break free of this trade off and have a continuous stream of releases of high quality.



Software Production Processes are Software too

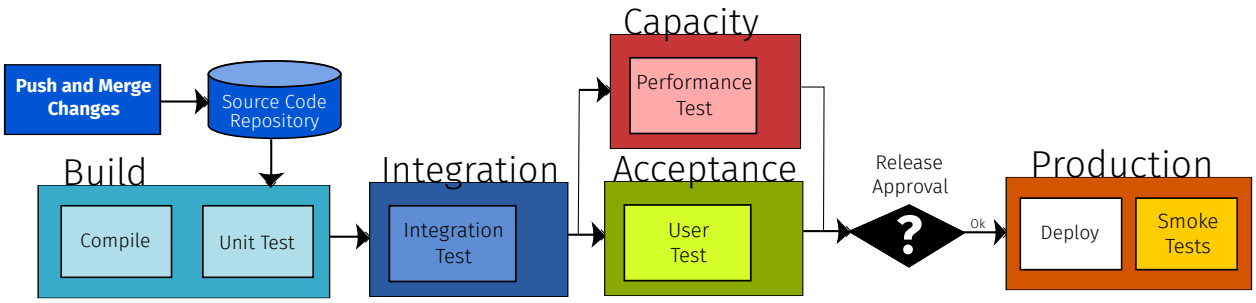
Let's go through the DevOps cycle once again: you start with the back log, prioritize it and make a plan for the next iteration. You implement the stories. You build your system, you write and run the tests. You package the release. Deploy it, install it and start it. While the system is running you monitor its behavior, gather feedback, try to learn something from observing the users. Get all the bug reports, put them in the backlog and go around once again.

This sequence of steps is what we call a process: because your development and operations team goes through a certain number of activities. When you describe a process, you are basically describing a piece of software. The question is: how can we automate this process? How can we write software that helps us to build, test, release, deploy, run, monitor software?

If you have automated such infrastructure, if you have tools which can support each of these activities, then the whole cycle can be fast, reliable and produce a high quality outcome. The only bottleneck that is left concerns the developers who need to write the code as well as the tests, but the actual execution of the other steps should be fully automated.

The cycle may stop from time to time: you do not necessarily want to make a change, build it, release it, and then wait until the user complains and then scramble back to fix all their issues as quickly as possible. That's why we want to have this little cycle on the development side in which you catch as early as possible all the problems, thanks to testing and test-driven development. If the test is red, we go back and fix the code and then test it again before shipping the release. The same is true also on the other side. Sometimes when you monitor some performance parameters you may be able to correct those issues by allocating more resources, by changing the configuration of your system, and those can be done directly in the operational side. You don't have to go back and make a change to the system to do it, especially if the system is configurable.

Software Production Pipeline



How can we design a piece of software that supports this process? What kind of software can enable such continuous life cycle? This is called a software production pipeline.

The input data to this pipeline is piece of software. It's the code of your software which gets processed going through serious series of stages. On the other side, out of the pipeline comes an image, a piece of software ready for deployment. This is an executable software artifact that can be put in so called production: it can be made accessible to your users so that they can work with it.

The software production pipeline transforms the software from its original source form into executable form, but it's not just a compilation step, that's only the very beginning. The build is actually not only compilation, but it's also checking that the unit tests pass. We want to make sure that all the components that we compile separately are passing the quality control.

Once you have built all the components you have to integrate them so you won't have to release and deploy individual components, but a single artifact with your software and as many of its dependencies as possible packaged inside. This can be subject to the integration test, another quality assurance step.

If this is green, further testing, for example, in terms of performance, helps to check that the system is not getting slower. So you compare the performance results with the previous release. And we run a benchmark to do that. And also you can already get some users to give them a preview and they tell you if they accept how the new features have been implemented.

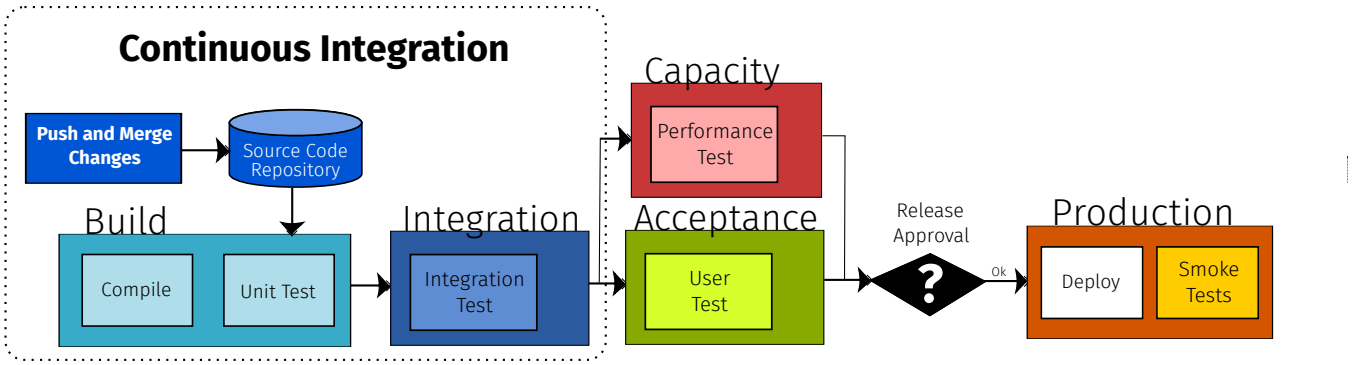
Based on the outcome of all these tests, the process reaches a fundamental decision: to release or not to release. That is the architect's responsibility: to approve the release, to sign off and state: "yes, we reached a state which we can publish our work and make it available to the users so they can start working with it in production". Once this happens, then you deploy the system in production and before you open the door to the users you still do a quick sanity check to make sure that everything is supposed to work as intended.

The most frequent word that you find in this slide is the word test. Software production pipelines emphasize automated quality assurance. At every step that you do, you need

some automatic way to check that your intermediate product meets expectations. You have to be able to encode these expectations so they can be checked automatically, both in terms of correctness, but also in terms of extra functional requirements.

How often do you run this process? Do you rebuild and retest every time a developer pushes a new change? Do you do it on a time basis? For example, you run it every night. How often do you meet and decide that we are ready now to make a new release? That's another decision that you have to take. How often would you like to potentially disrupt the production environment? In some conservative companies, for example, they say that they freeze their systems in November, because the last part of the year is critical from a business point of view, and they don't want to risk the negative impacts of any change. Either you make a release before November or you wait until the New Year and stop the last part of the pipeline during this window. You can still work over in the first part of the pipeline, but you cannot access production. That's why it's important to take responsability for the decision to ship a new release.

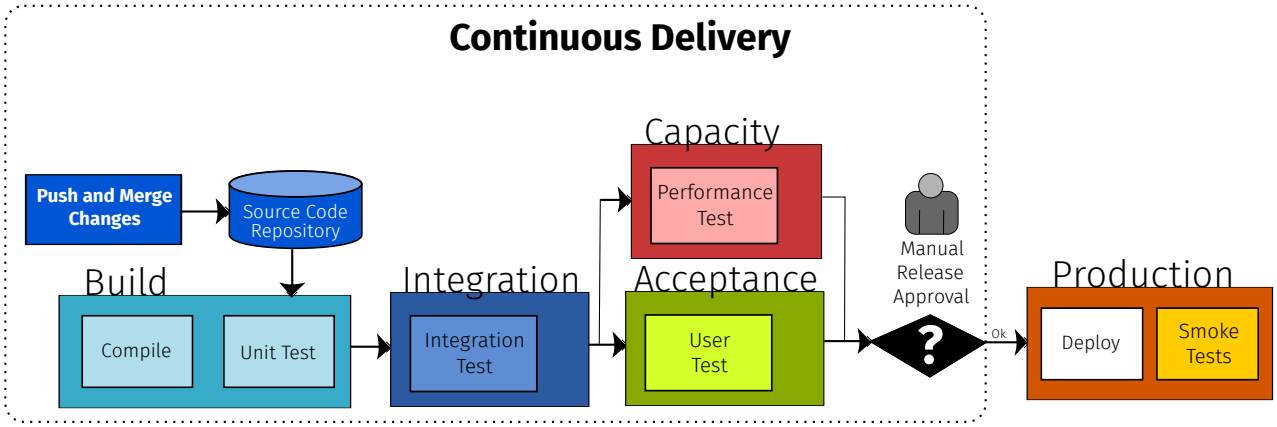
Software Production Pipeline



Frequent commit, push and merge of changes with automated build with tests (At least once per day)

We use the term continuous integration to refer to the first part of this pipeline. Developers commit, push and merge changes into the build with a certain frequency (e.g., at least once per day). The pipeline will pull the changes from the repository and go as far as building, testing and integrating the changes with the rest of the system. The result is something that we can run. There is a working version of the system that has passed unit and integration tests.

Software Production Pipeline

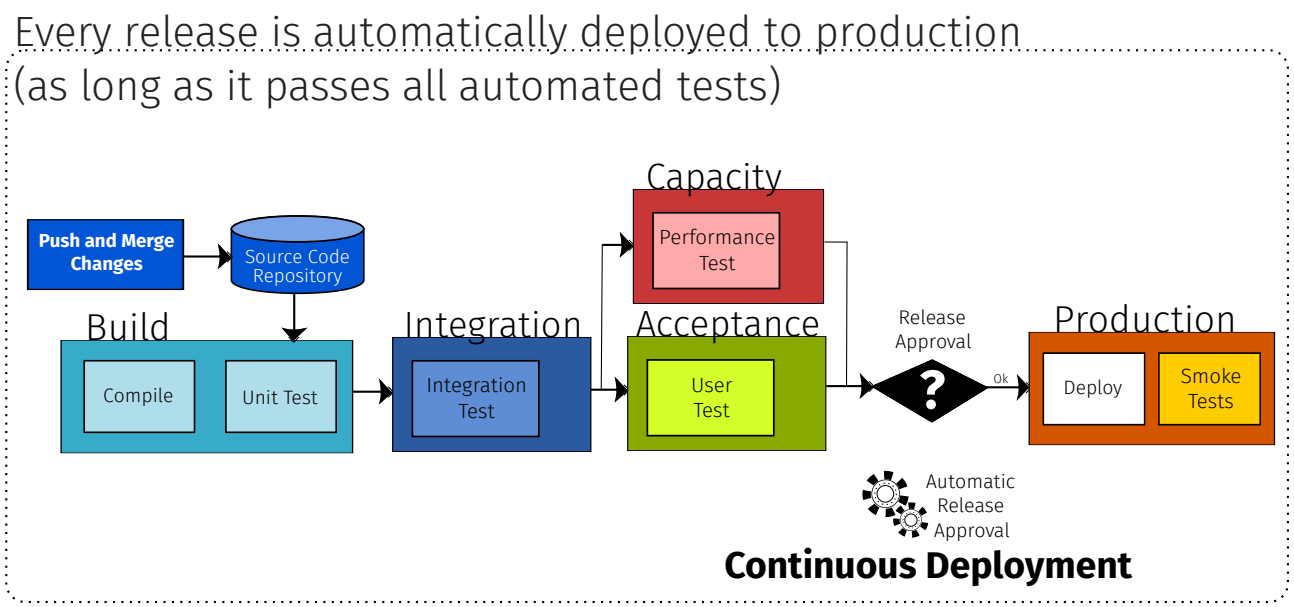


Frequent automated packaging of releases, fully tested and ready for manual deployment in production

Continuous delivery goes one step further. In this case we obtain not only something that is ready to be executed, but it's also fully tested. It passed all the tests that we have and it's packaged ready to be released for manual deployment in production. If we do continuous delivery we go all the way until the decision step, in which somebody has to take the decision. And then if the decision is yes, we are ready to start the deployment in production. The deployment itself can be manual or automated, but it will only start if someone takes responsability for it. This is difficult if you want to deploy new releases every second.

During the compilation step, the compile targets a certain runtime platform. The goal is to run it in an environment that is as close to production as possible. If your users have different platforms (e.g., different operating systems) then all of your build and test pipeline have to reproduce those platform. So you will need to do a build for Windows OS, Mac, Linux, Android and iOS. The infrastructure to do this gets rather expensive because you need to instantiate all of the pipeline stages for every one of those platforms, and especially for running tests. The platform is defined not only by the OS, or the Web Browser, but it may also include other configuration parameters, describing the assumptions your architecture makes on the environment in which users will run your system. For example, different screen sizes, input/output devices. If you want to run a systematic test of how your responsive user interface adapts to all of these different platforms, browsers and device screens, the size of the configuration space to test will explode.

Software Production Pipeline



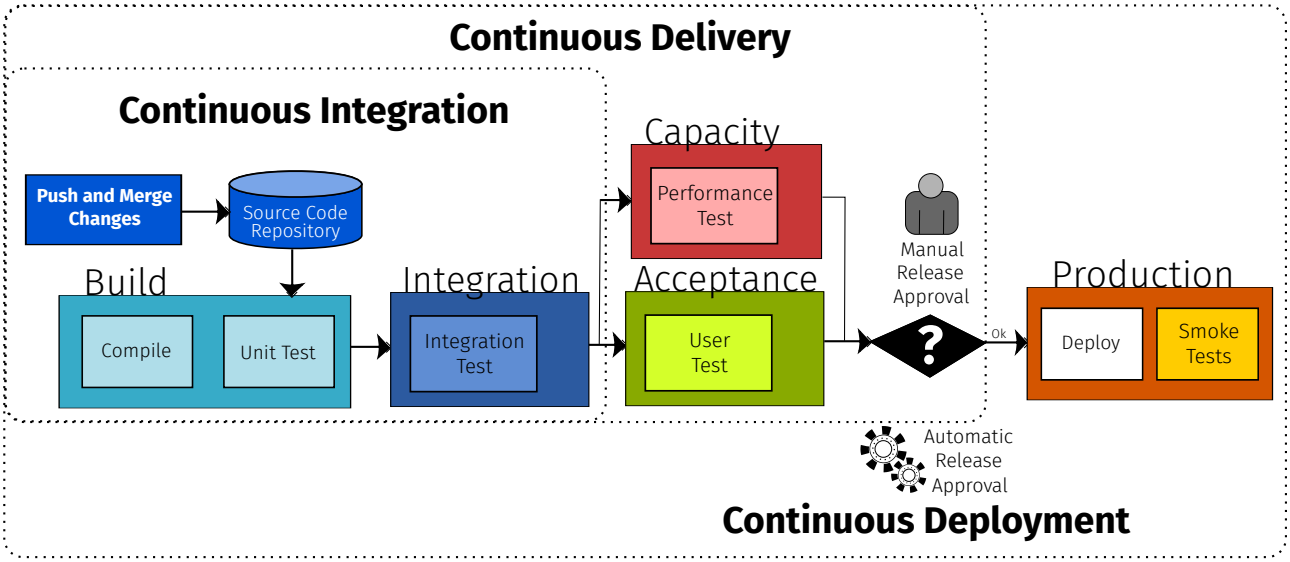
If your goal is to continuously deploy, then you have to get rid of this manual decision point and also make the decision automatic. You need to write some rules that determine that as long as our new version passes all the tests, it will get deployed automatically. Such approach puts a lot of trust in the quality of your automated quality assurance tasks. There should be a very extensive coverage and you need to really trust your tests actually mimic the conditions under which your users will work with the system.

We can also break the decision in two steps. On the provider side, you approve the system as being stable and ready for prime time. On the consumer side, you decide if you want to accept the risk deploying the release. This also depends on the kind of runtime environment is targeted by your architecture. If you split into the classical Cloud/Mobile, you can continuously push to the Cloud, but there will be a significant delay before mobile clients will receive their side of the release, both due to AppStore approval processes (those are not automated and can add significant delay) as well as mobile phone users who give a low priority to keeping their apps up to date.

Do you really want to have automatic updates on your mobile OS and the apps deployed on it? If you accept that, you trust who is making the release decision. You assume that if they send you an update, this is actually an improvement, in your best interest. For example, if you want to get the attention of users and nudge them towards accepting the update, you should always tell them: it's a security update. If you don't update your OS, you're going to get blamed if you get hacked because you didn't install the security patches. What if instead the OS update would slow down your hardware to increase the likelihood that you buy a new phone sooner? In the long term, the big question is how much control users are going to have over "their own" hardware. Ownership, transparency and control of hardware devices is becoming questionable, since a lot of software running on "your own" computers is deployed without full knowledge or awareness of the end users.

Overall, automated release approval and deployment acceptance gives you the performance that you need to avoid blocking the software production pipeline because a committee has to meet and agree that your software can be shipped. If you have a release that is automatically built, tested, released as well as deployed, you joined the Brave New World in which every time your users connect to your system they are potentially using a brand new system. Probably it will not be completely different than the one they used the day before, because the changes that you introduce are small, but over time these changes accumulate. Congratulations: your software has reached the age of continuity.

Software Production Pipeline



To summarize, continuous integration is developer oriented. Developers already benefit from this because they have a safety net as they make changes to the code. The pipeline will reliably produce integrated artifacts that they can already test and execute in their environment.

With continuous delivery we extend continuous integration with a manual decision to make a release. The decision is based on the information gathered during the automated tests.

Continuous deployment extends continuous delivery automating both the release approval decision as well as the upgrade process to deploy the new release in production.

Which is the most common option? It depends on the maturity of the organization and also on whether they trust their tests to give them sufficient information to make the decision automatically. When you introduce continuous deployment also the users need to be aware that they cannot stay behind and need to be exposed to a continuous stream of changes.

High Quality at High Speed

- Every team member can see and influence the build process outcome
- Any pushed change can be released at any time
- Small, frequently committed changes are less likely to conflict
- Keep the build fast (Get feedback as early and quickly as possible)
- Build after every commit, build every night (and run more tests)
- Reliably and reproducibly build any version of the software (Keep a snapshot of all dependencies)
- Released images are immutable

If you work with continuous integration, delivery and deployment of your software, you follow these best practices as you work at high speed to produce high quality software releases.

Every team member has an impact and visibility into the build process. This is about making everybody responsible and giving them the possibility to fix the problems that they introduce. It should not be somebody else's problem to watch over the build, but everybody has to keep it green.

It's a big responsibility if you push your changes into one of these pipelines because, potentially, your commit – if it passes all the tests and there is an automatic decision – can end up in a release anytime. Developers should be aware about the impact of their changes, keep the changes small and commit them as frequently as possible to minimize conflicts with other developers.

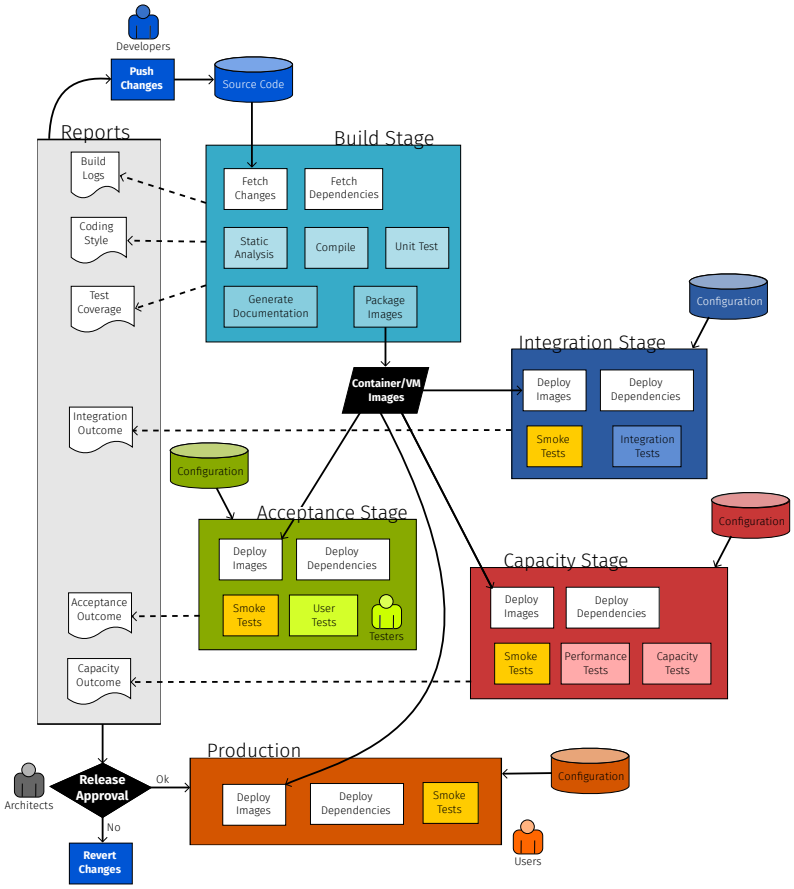
It's important to keep at least the early stages of the build very fast because when you push a change and you make a mistake you want to see that the build breaks and you want to get this critical feedback as soon as possible. So that you can still have fresh in your mind what exactly did you touch and how to undo that problematic change.

Depending on how expensive it is to do run the build, you should do at least an incremental build after every commit. During the nightly builds, since there is more time, run more tests to extensively check the system as opposed to after every commit when you want to give an outcome as fast as possible. How are nightly build handled when you have developers across the world? If the sun never sets on your software development teams, you need to just make sure you regularly schedule a full build and only some of your developers will get the results when they wake up.

The build process should be not only reliable, but also reproducible. Based on the code repository, on your versioning system, you should be able to build any version ever released in the entire history of your software. The outcome should be the same no matter

when you run the build, even years later. As a consequence, you do not only want to version your code but also take a snapshot and version your dependencies as well. This is a typical beginners mistake: you do not version your dependencies and then years later when you try to build an earlier release to reproduce a bug, you're no longer able to do so because the dependencies have changed and you cannot access the old versions of the old dependencies that you used in the old release. Once you depend on something you want to grab a copy of it, and store it in the repository together with your own code. You want to keep the copy around because dependencies change as well. And your code is not necessarily forwards or backwards compatible.

Likewise, once you make a release, you tag it. You don't change it anymore, so that the version number and the build number refer to an immutable artifact. While there is always the temptation to go into your Docker image and make a few quick changes, try to avoid doing that. You're doing something manual and the whole process is supposed to be automatic. In the same way that you typically do not edit the bytecode or assembly code produced by your trusty compiler, if you want to do those small, last-minute corrections, you do them to the input source. Then you make a new build, you have a new image with the new build number.



The software build pipeline can grow to become a relatively complex piece of software on its own. There is a whole set of products that software development companies use to actually automate this whole process. We start from the top where developers push changes into their code repository. From here the software becomes a piece of data that has to be processed, checked and transformed. Eventually it flows down towards production, where we have the users. Before the final deployment in production step, we have to make a decision to whether to approve the release based on all of the artifacts, metrics, and reports which are the outcome of the build and the tests phases.

The first stage is about building the original source code. Building it involves a compilation step, but there can also be a static analysis step. This is also where one can generate documentation. Unit tests run after a successful compilation. Their coverage can be measured. And since these tests are focused on each individual component, mocks help to isolate the component under test. As a result, you have packaged your component as a deployable artifact: an image.

What happens to the artifacts obtained along the pipeline? They get stored in an image repository with a certain version number called the build number. The process works with these images, checking they can be successfully deployed. In the integration stage, components as they get combined together with their dependencies will be further tested. Finally, it becomes possible to run an end to end test with the complete system, including all of its dependencies.

If the integration is successful, we move on to the acceptance testing where sometimes you can automate this, but sometimes you have a dedicated crowd of test users. The goal is to observe the end to end behavior by stimulating the system from the user interface. Different use case scenarios can be validated to reproduce real world usage of the system.

At the same time, the capacity, performance, or even scalability testing stages check how many resources are required to achieve a certain level of performance. Here the

focus is on the extra functional quality attributes. Acceptance reports the system works correctly, here you care whether the correct result is delivered on time, given the available resources. The capacity stage deals with the question: how much processing power do we need to allocate so that the system meets the performance targets? Can we estimate how much does it cost to run the system under a given workload?

If all of these outcomes are green and the architects (or the automated release approval rules written by them) agree that this is a good release, then it is pushed in production.

If there is a problem with the build, you have to fix the broken build. One way to fix it is to undo the changes that broke the build. So this is always a possibility. If it turns out that it was not possible to correct the problems, then you go back to the latest known state in which the system used to work. And you restart again from there.

Why do you still need to do testing even after you deploy your system in production? Because there should be an automatic way to check that the system is successfully initialized and to assess whether it's available: did it start correctly? Is it ready to work with the users?

Such pipeline looks complicated and also very expensive to set up. Remember: software to build software is still software, make sure you at least use version control to keep track of how you change it, if not a fully fledged production pipeline to build your main pipeline.

You could try to get away with a simpler pipeline. Maybe it's enough if you build your system: compile it, package it, but once you have an image, just send it to your users, skip all the time consuming testing. This is called testing in production. If you put the new version in production, the user will be in charge to report whether it works or not. If it doesn't you will hear loud complaints from the user soon enough. This is a form of outsourcing the quality assurance for your software to your user community. If users are willing to pay you to become your testers, then you can consider skipping most of the pre-release testing. Clearly, users should be aware they are using partially tested releases (sometimes these proudly sport a greek letter in their name: alpha, or beta). Otherwise your reputation will suffer.

In every piece of software, testing in production is always the case. Only in some cases one can afford to also do unit, integration, acceptance testing and try to avoid pushing out bad releases.

Types of Testing

Unit	Test individual components immediately after they are built (if necessary mocking their dependencies)
Integration	Test the complete system (integrated with all dependencies)
User	Test the end-to-end behavior from the user interface (check use-case scenarios work) - only partially automated
Capacity	Observe whether extra-functional quality targets (e.g., performance, scalability, reliability, capacity) are met to determine the resources required for a given workload

Types of Testing

Acceptance	Test the system in an environment that is as close as possible to production to determine whether or not a system satisfies the acceptance criteria and to enable the customers to decide whether to accept or reject the new version of the system.
Smoke	Quickly probe the system to check its successful initialization and state of readiness and availability

Note: Every company performs **testing in production** with end-users, some companies can afford a dedicated staging environment for running all of the other types of tests before shipping a release

Types of Release

- **Big Bang/Plunge**: Everyone switches over to the new release after some down-time (with high hopes and no way to go back)
- **Blue/Green**: Everyone switches over to the new release at once and can revert back in case of problems
- **Shadow**: The release is deployed but not used (Dark launches test the release with real user input without revealing the output)
- **Pilot**: The deployed released is evaluated by a few users for a limited time

How do we make different types of releases?

The original, oldest type of release is the Big Bang release, where we do it in one shot: every user goes to the new version. Deployment takes some effort, produces some down-time, some disruption because of the release requires to first switch off the existing system before the new one can replace it. Hopefully the new one will work. It must work, as there is no way to go back in case it doesn't. The Big Bang release is a high risk proposition, which you want to avoid.

If you have a good deployability of your architecture, you can support other types of releases.

A blue/green release still involves switching as atomic step: every user will go to the new version. But in case there are some problems, you still have the old software deployment around and you can easily switch back and forth between the two. This already gives you a very important advantage, which is the ability to undo a failed release. This however makes it more expensive because you need both the blue system and the green system. So the number of virtual machines that you need doubles. The old ones should be kept immutable. Once the new ones are ready, you switch. If everything works fine, then after a certain time you can stop and decommision the old. If the new version doesn't work, you can go back.

As opposed to the blue/green release where all users will see either the new or the old system, if you do a shadow release, you make the release but you don't use it. Why? Why do we go to the trouble or building the system, testing and releasing it and then we don't give users access to it? What you want to do really is to connect the new release to the input of the users so that the system is actually undergoing the production workload. But you don't show the output to the users, yet. Users still sees the output from the old version, since the new one is still kept in the shadow. If you do so, you have a chance to observe and compare the output. And if the output is different and you don't expect it to be different, maybe you just detected a problem. Shadow releases are subtle way to let real users test your system with the real production workload but without affecting the users in case something goes wrong. Once you are confident that the behavior of the new version doesn't have any regressions, you can bring it out of the shadow and make also its output visible to the users.

Another technique is to do a pilot release. When you make a change, you deploy it, but you only make this visible to a small set of pioneer users. The users are going to give feedback and then you can decide if it is worth to extend the access to this new feature to the rest of the user population. Here the main challenge is: how do you select the users that are willing to try the new version? You need to have a set of trusted users who provide constructive criticism and – in case something goes wrong – they will not blame you. One way is to call for volunteers. Have you every started an app and there was a pop up asking you to try to switch over to a preview of the new version? You could have been randomly selected. Or maybe they observe your interactions (or your poor reviews) and they see that you could potentially benefit from these new features so chances are you're willing to give it a try.

Types of Release

- **Gradual Phase-in**: The deployed release is used by a gradually increasing number of users. Requires to support multiple active versions in production.
- **Canary**: A gradual phase-in release with very few initial users who do not mind dealing with failures
- **A/B Testing**: Roll-out multiple experimental versions to different subsets of users to pick the best version to be eventually be used for everyone

There are also releases in which the switch between old and new version happens gradually. To do so, you run multiple versions of your system in production, and gradually spill over more and more users to the new version. The pilot is just with a small set of users and you're not sure yet if you will go ahead with every user. After a successful pilot, you can gradually get more and more users on board. This way you can increase your confidence that the new version is actually stable and powerful enough.

Canary is a similar term for a pilot (the term comes from the Canary in the coal mine). In this case you make a Canary release because you don't trust its quality but you make it accessible to users that are aware of this. Still, they don't mind having a few crashes because they know they're working with an unstable system.

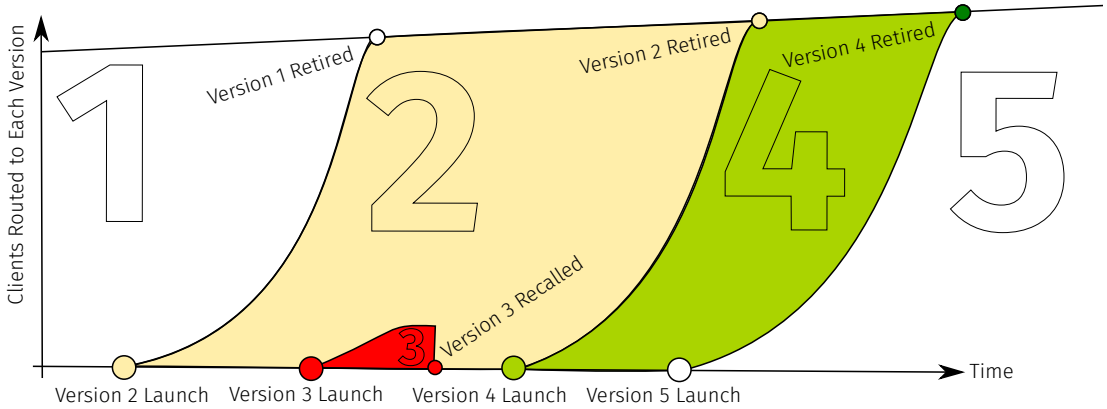
A/B Testing is also related to the concept of having multiple active versions in production. It is used if you do not know when you make a change, whether it is actually an improvement or not. You make multiple experimental versions available to different users. Based on some metric (typically this is some kind of monetary metric), you compare which version outperforms the others. For example, which color of the links makes people click on the ads so that we can make more money? That's one of the original use cases for this technique.

To support these advanced types of releases, the infrastructure for your continuous

integration does not only contain all the quality assurance phases that we have discussed, but it actually becomes much more complicated because you have to collect feedback metrics that you measure from the user behavior in production. Then you need to feed them into some statistical inference to see whether your experiment is validating the hypothesis that version A is better than version B or not. You can afford this when you have millions of users coming to your system every day. Very quickly you get the results necessary to support your experiments. If you just have 10 users it's very difficult to draw any conclusions by passively observing their behavior.

Related to this is also this idea of making releases which have partially implemented functionality. For example, you show buttons on the screen. But you don't implement the corresponding behavior of the application. If the user never clicks on these buttons, it means that the feature is not wanted by your user population. What's the point of investing in the implementation of a feature that nobody is interested to click on? If it turns out that people are actually clicking on the button, then you show them a little message that says: "Please sign up to be notified when we release this new feature". If they sign up, they are really committed and you have a chance to deliver to them a rough implementation as quickly as possible. But now you know that some users are actually interested about it. Now you can justify your investment into building it, as opposed to building it in the hope that users will come. You will be surprised about what you can learn from this: it can completely change the way you prioritize your backlog.

Gradual Phase-In



Here is a visualization of the gradual phasing type of release. On the vertical axis we have the number of users for each version and horizontally we have time.

At the beginning we have version one in production. And every user is using this version one. Then we launch the new version, but we don't do a Big Bang release where every user switches over at the same time. Instead, we actually have a gradual phasing were initially only small number of users have access. As we observe the logs, we can conclude that it doesn't crash, users seem to be happy with it. There are not a lot of bug reports, so we gradually move over more and more users. Until version one gets retired: nobody is using it anymore. So we might as well switch it off.

These transitions can take some time, especially if you leave a choice for users whether to upgrade or not. While we're still in that first transition, we attempt a pilot for version three. We launch it, but very soon discover that it's not a good one. There is no acceptance by the user that have a chance to look at it, so you quietly pull it back and switch back every user to the previous version 2. Since version 3 was only seen by a small minority of the user population, the failed pilot is not disruptive. Version 4 happily works better, so gradually every users will be migrated to it.

Depending on how each transition takes, you might end up for a given point in time to having to maintain multiple versions of your system in production at the same time. This is more expensive than a simple istantaneous switch. You have to ask yourself how many of those can you afford to keep running, and this will naturally give you a limit on how often you can make new releases depending on how long does such transition takes.

Essential Continuous Engineering Practices

1. Automate everything
2. Push changes regularly
3. Don't push on a broken build
4. Commit only if locally green
5. Watch the build until it's green
6. Never go home on a broken build
7. Never comment out failing tests
8. Always be prepared to revert
9. Time-box fixes before reverting
10. Direct changes in production are forbidden

Now that we have seen both how to build and release, let's conclude the continuous delivery discussion with a number of best practices that you may want to adopt into your development toolbox from now on.

Make sure that the process that you use to write your software and test it is automated: don't try to do things manually because you wouldn't do them anyway and if you did, you would be slow.

Push your changes regularly. That means at least once a day. You attend the stand up meeting in the morning, you work, and then before you leave you should push.

If the build is broken, you shouldn't push more stuff on it to break it even more. Before you push, make sure that your local build is green. Never push something that doesn't compile, because then you would break the build for everybody else.

After you push your change, don't go away. Watch the build until it is green. Also, never go home on a broken build, so if you break it, it's your responsibility to fix it. In the worst case, you just have to undo the work of the day and tomorrow it will go better.

A big temptation for fixing a broken build is to remove failing tests. You can always blame the tests and not your code. My code is good and these tests are obsolete so might as well comment them out. If you do that, you reduce the coverage and of course then you get a green build, but it will be a fake green build (just like when someone recommended to slow testing down to improve the pandemic statistics, but I digress). If the tests are failing and you blame the test, then fix the test. Don't just skip them. How long will you need to stay to fix a red build? Some developers never go home. You should time box it: if there is a problem with my latest change and I cannot fix it within one hour, then I can revert the changes. The work of one day has disappeared, but I can go home to rest.

There is always the temptation to fix problems directly in production. You forgot a

comma. You open into the config file, you edit it. And the lights are on again. But this is not enough, because where did the missing comma come from? After rescuing the day, don't forget to fix it at the source. And then trust your tools to smoothly produce a new release, push it in production and everything is fixed. It might take a little bit longer, but the problem will be fixed for good and not come back to haunt you again the next time.

If you choose to adopt this distilled wisdom, then you are actually practicing continuous software engineering.

Tools

Continuous Integration

- Cruise Control
- Jenkins
- Travis
- UrbanCode
- GoCD
- Packer
- GitHub Actions

Container Technology

- chroot
- Docker
- Docker Compose
- Docker Swarm
- Kubernetes
- Mesos

Build Pipeline Demo Videos

Explain how to setup automatic software production pipeline for your tool:

- How do you build a component?
- How do you test a component?
- How do you plan integration testing in your pipeline?
- How do you plan capacity and acceptance testing in your pipeline?
- How do you package a new release?
- Which types of release is supported by your tool? Demo at least one type.

Build Pipeline Demo Videos

Deployment:

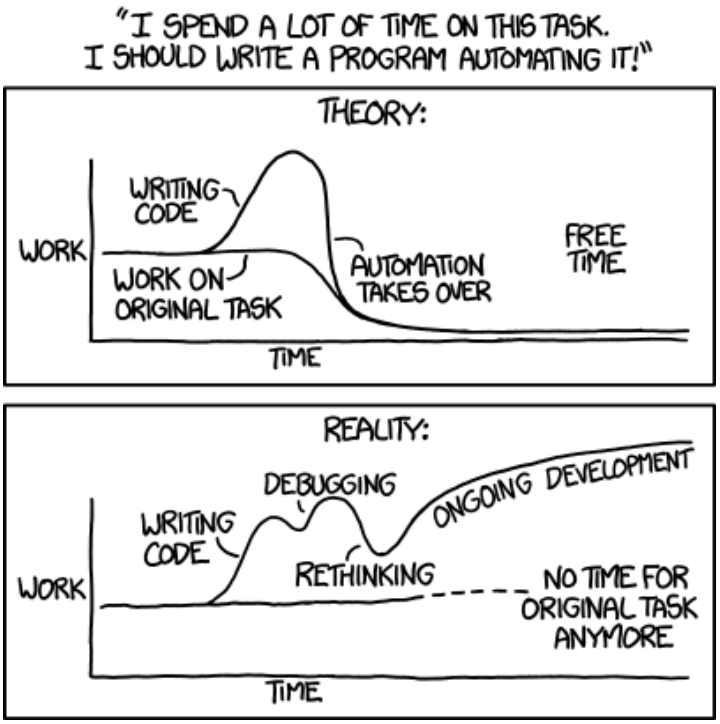
- How do you deploy an application?
- Can you automatically deploy the application if the tests are green?

Failure Scenarios:

- What happens if something goes wrong in your build?
- Does the tool provide reports or logs after each phase?
- Can you show different examples of a broken build and how to fix them?

Container Orchestration Demo Videos

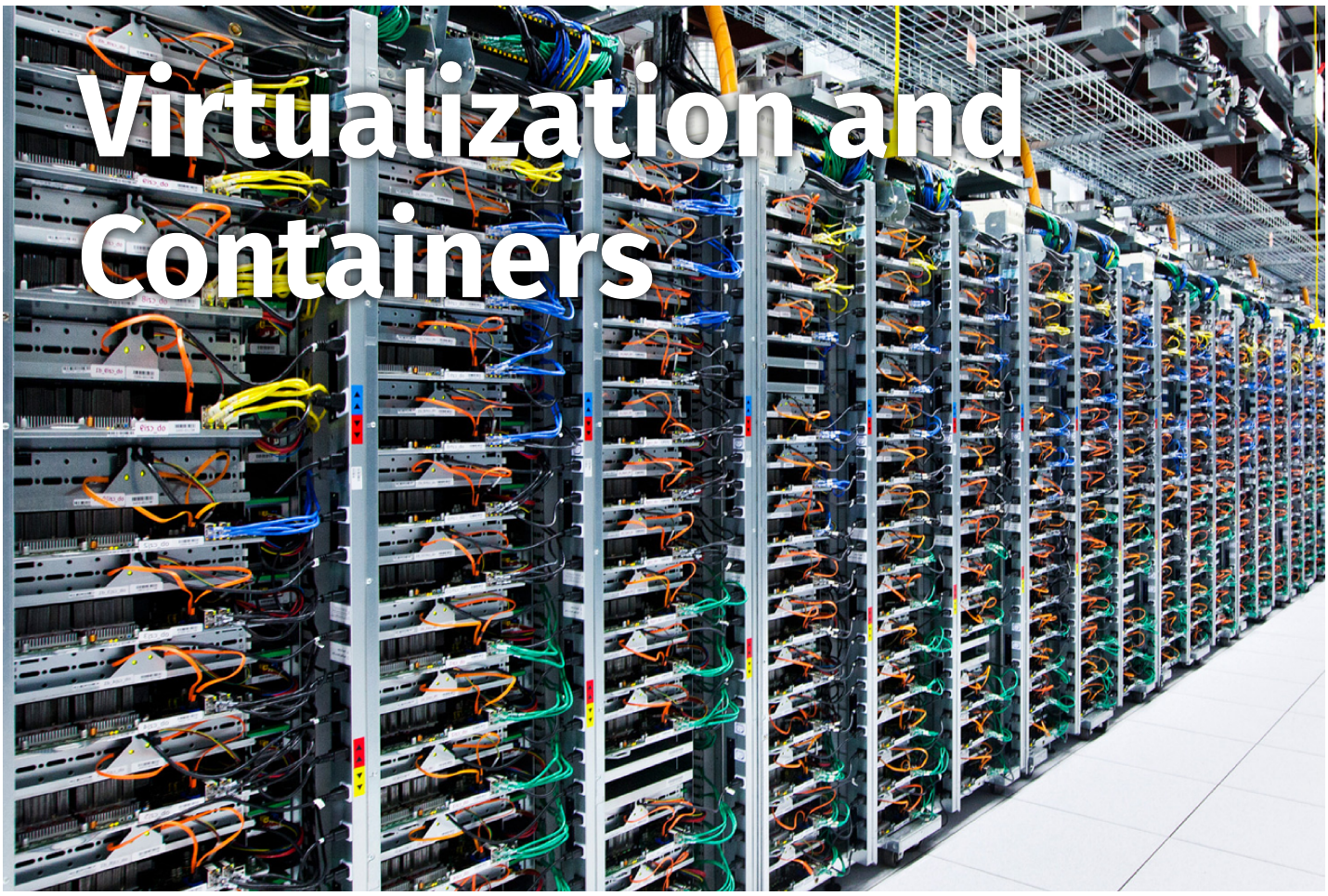
- What is a cluster? How do you create a cluster?
- How do you configure Kubernetes?
- How do you create a deployment?
- How do you create nodes and pods?
- How do you open an application to the users?
- How do you create multiple instances of the an application?
- Which types of release is supported by Kubernetes? Show at least one.



Today's XKCD is about the benefits of automation. Should you just get something done or invest into automating it? When you expect that you will do it very frequently, it may be worth to invest some time and effort into automating it so that eventually you will have more free time. This is also our ultimate goal too: let your automated build pipeline work on your release. In theory, automation takes over and your life is better.

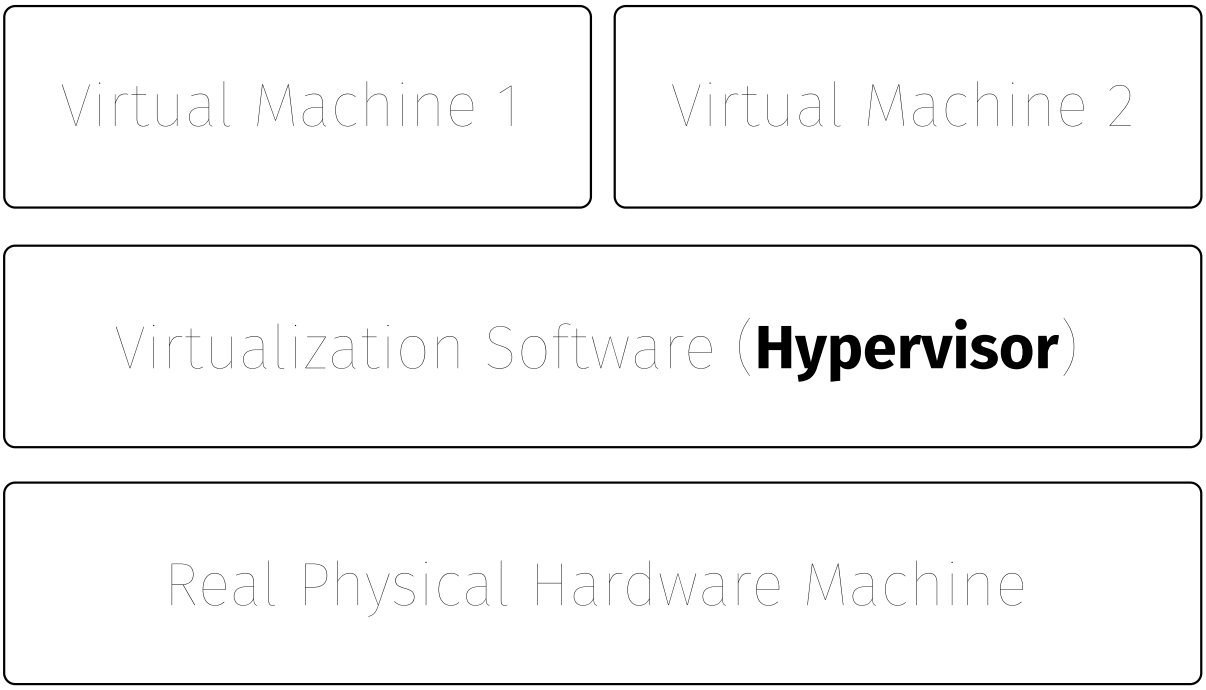
The reality: it turns out the software that you have to develop for running the software production pipeline needs to be written, tested, debugged, or it needs to be selected, deployed and configured, and tested and debugged. As usual the development never ends: instead of having more free time, you don't have any time left for the original task.

But what if you could automate the writing of software production pipelines as well? Before we jump into infinite recursion, remember the lessons of how to bootstrap programming language compilers and see if you can apply them to this other problem.



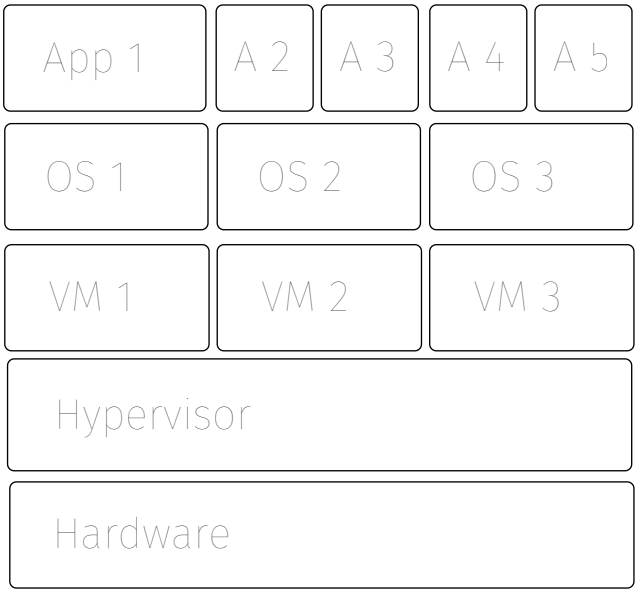
Virtualization and Containers

Virtualization



The enabling technology for deployability is virtualization. More recently virtualization evolved or specialized into containers. Virtualization is about isolating your code from the actual environment in which it runs, like when you deploy it up in the Cloud.

Virtualization



Hardware made into Software

Hypervisor: Intermediate software layer that decouples the above software stack from the underlying hardware

Virtualization can be applied to different hardware resources (CPU, memory, storage, and network)

At the bottom we have the actual hardware. On top, we place a layer of isolation: the hypervisor. On top of it, we obtain virtual machines on which an operating system can run your applications. From the perspective of your application, the virtual machines are like hardware, but they're actually simulated or emulated by this piece of software, itself running on the real hardware.

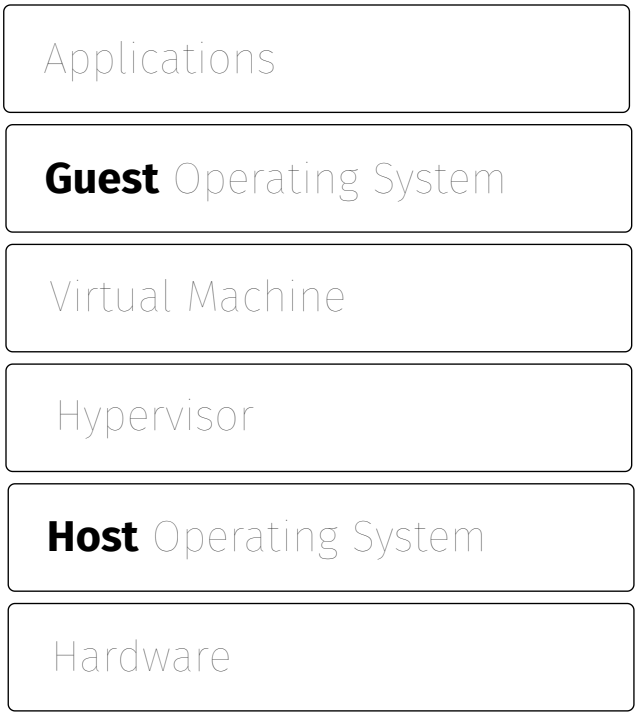
This isolation layer makes it possible to deploy any application on any physical runtime environment, since the application is decoupled from it. Clearly, there is a cost. The larger the gap between the actual hardware and the one emulated by the virtual machine, the more expensive it will be to run the application with a sufficiently good level of performance.

Even if the virtual CPU and the actual CPU are the same, the other important goal of virtualization (which is also shared with containerization) is to isolate the various application and their operating systems from each other. If you run an operating system directly on the hardware, all the applications or processes in this operating system are both aware of one another and can somehow interfere with one another. This may be a feature, since they can use, for example, the shared memory or shared file system to exchange data. This may be a security issue, if the applications run on behalf of different users, who may be business competitors. Running these conflicting applications on separate virtual machines guarantees isolation between them, almost if they would be deployed on physically distinct machines.

Other advantages of virtualization include so-called elasticity: How much of the underlying physical hardware should be dynamically allocated to each of the virtual machines? Depending on the workload of each application, the virtual hardware can be made more or less powerful.

So in other words, as we've seen many times in software architecture, adding some intermediate software layer helps to decouple what is above from what is underneath. Let's see different examples of how we can play this layering game.

Virtualization



Hosted hypervisors require an operating system to run

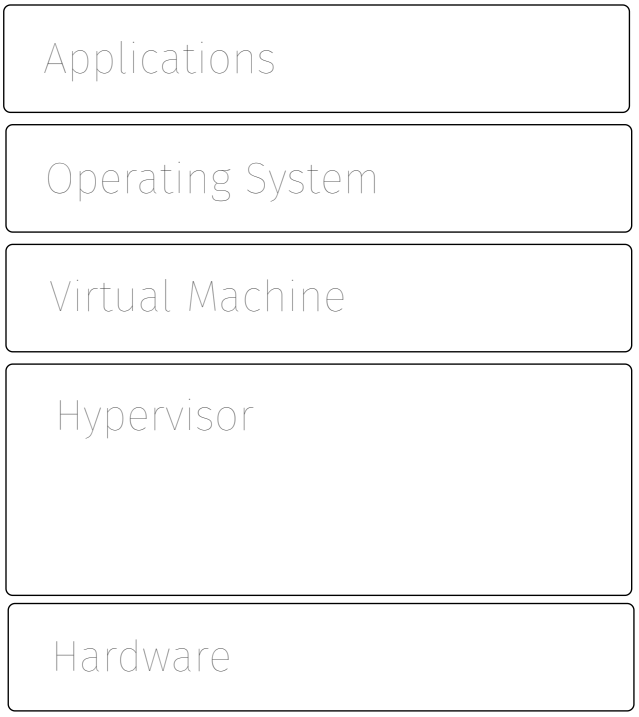
Example: VirtualBox, VMWare Workstation

In this scenario, we have two operating systems running: the host and the guest. The host runs directly on the actual physical hardware. From its perspective, the hypervisor is an application. Within this application we can run simulated hardwares on which we are going to have the guest operating system that will run our own applications.

This was one of the early virtualization approaches. This is also what you still do today when you run virtual machines on your computer. For example, you have the MacOS operating system running on the Apple hardware. On top of it you can run Linux, Windows, or whatever other operating system. This is great because the applications which depend on Windows can still run on the Mac. From the point of view of the application, they are not aware, but it can be convenient for users that cannot afford to buy separate computers to run different operating systems. For some reason, the opposite scenario, running MacOS in a virtual machine on top of a different OS, is not so common.

Here we also have many layers and the performance suffers. Let's try to optimize this architecture.

Virtualization

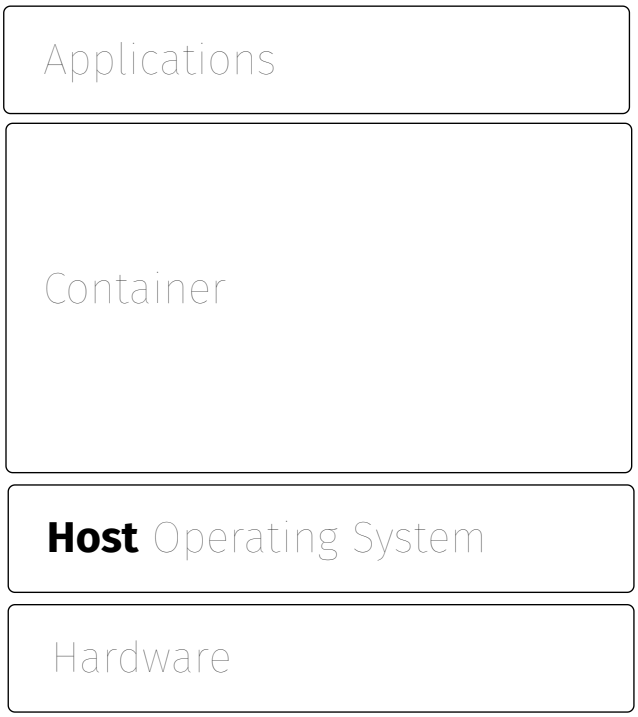


High performance, native hypervisors run directly on the hardware

Example: original IBM Mainframe z/VM, Xen, VMWare ESX

What if we blend the bottom layers? If we decide that the only purpose of this particular piece of hardware is to host virtual machines, then we do not need a fully fledged operating system on the hardware. We can delegate the role of the hypervisor to just abstract the hardware and use it to offer virtual machines that we can use to run multiple (guest) operating systems. This reduces the redundancy in the stack, and can be optimized for the specific purpose.

Lightweight Virtualization with Containers



Containers are lightweight solutions to isolate applications that can share the same operating system

Example: Docker, rkt, OpenVZ, Hyper-V

The other approach comes from the opposite side. There are lots of layers between the application and the hardware. What if we can assume that all applications will run on the same guest operating system and this happens to be the same as the host? Then the goal becomes to isolate them as if they were running in separate virtual machines, but without the overhead.

This is why containers are sometimes called ”lightweight virtualization“ within the same host operating system. If you can spot the difference, the applications are directly running on the host operating system, but the applications and operating system are separated by the container. This layer provides the needed isolation without the overhead of having a hypervisor underneath.

	Virtual Machine	Container
Footprint	GB	MB
Boot Time	Minutes	Sub-Second
Guest OS	Different in each VM	Uniform for all containers (Linux)
Deployed Components	Multiple, Dynamic	Single, Fixed
Isolation	Hardware	Namespace OS Sandboxing
Persistence	Stateful	Stateless
Overhead	High (with Emulation)	Low (Native)
Baking Time	Slow	Fast

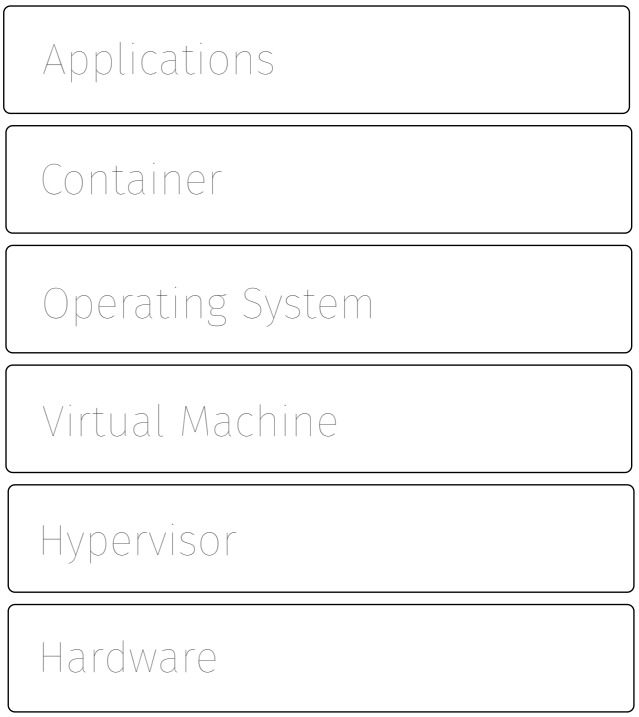
So which one is better? Containers or virtual machines (VM)? In terms of resources, the term lightweight virtualization is justified for containers. Virtual machines need to simulate whole hardware environment in which to run a full operating system: they consume gigabytes of memory. The footprint of containers typically depends much sooner on what you will deploy inside, since they have a smaller overhead.

How long does it take to start a virtual machine? You have to boot the emulated hardware, you have to start the operating system, finally you have to wait for your application to start. Depending on the operating system and application, this may be a relatively slow process. If you have a container, the operating system is already running, you just have to initialize the container and start the application. This tends to be much faster.

What about security? What about the isolation? If the virtual machine has full control over the hardware running a dedicated hypervisor, it could take advantage of specific processor features to achieve isolation. With containers, it is questionable whether two applications deployed in separate containers sharing the same operating system are really as isolated. Namespacing and other sandboxing techniques pretend that processes running on the same operating system are inside a separate container and therefore should not detect the presence of each other nor interfere with each other. Ultimately you get what you pay for.

Baking refers to the process to build the images which can contain a whole operating system, including the whole application and its data. This is a slow process when building an image for virtual machine. With a container this is relatively faster because the image simply needs to store less bits.

Containers inside VMs



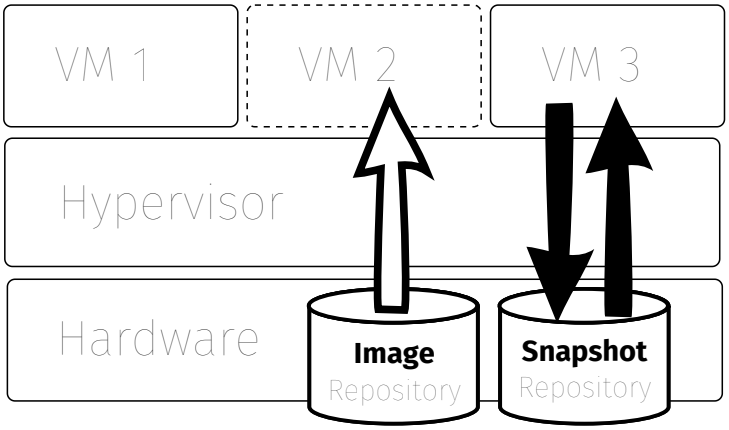
Containers and VMs can be combined to get the best of both worlds (however performance will suffer compared to raw iron)

Example: VMWare vSphere integrated containers

Good ideas can be combined with each other. Typically one needs to pay a Cloud provider to rent virtual machines. If you choose an infrastructure as a service provider, you get billed for the time during which your virtual machine is running. The more applications you run, the more expensive it gets because each should be deployed in a separate VM to keep it isolated.

Still, within a VM you can run whatever operating system you want. What if you install docker inside a virtual machine? Then each application can be deployed within its own container and you just need to pay for one virtual machine, which needs to be allocated enough capacity to host all containers. Also in this case, the operating system can be optimized for the specific purpose of hosting containers.

Images and Snapshots



Virtual Machines or Containers are booted using a preconfigured image selected from a repository

Snapshots or checkpoints of the entire virtual machine state or the container file system can be saved, backed up and restored

What are these images in which software component releases are packaged as?

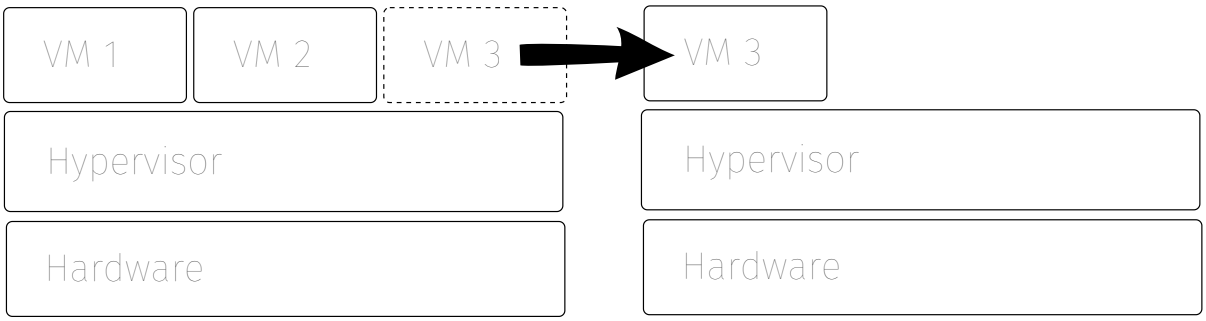
While simulating or virtualizing hardware, it becomes possible for the hypervisor to take a snapshot of the state of execution of the virtual machine. This includes a copy of the whole memory, a copy of the processor state. This snapshot can be written into a file. By reading it, it is possible to restore the state of the virtual machine, which can continue running from a known state.

This uses the same techniques that your laptop uses to save a snapshot of your system state just before it runs out of battery. The laptop is going to take all the volatile information, write it out the disk so that when you plug it back in and you switch it on, it will restore the running applications and operating system from the image.

We can benefit from this technique to provide an image from which we can start our application in a known state. This image is prepared (during the image baking process) by creating a virtual machine, installing the operating system as well as all of the applications. We start them and then we freeze it. Later we can copy it and deploy where we want to run it. Since it's just a matter of initializing the virtual machine from the given image and all the necessary software is already installed and ready to go.

A similar process is used for container images as well.

Virtual Machine Migration



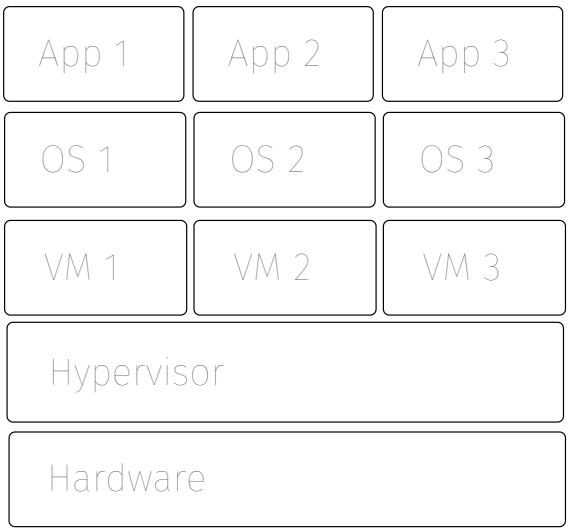
Virtual Machines can be moved between different physical hosts for load balancing, workload consolidation and planned hardware maintenance

Some hypervisors support live migration during which the VM does need to be stopped before it is restarted elsewhere

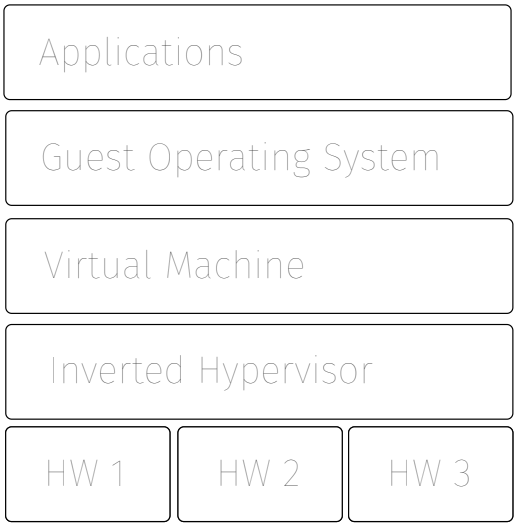
Virtual machines also support live migration from one hosting environment to another. In case the hardware fails, we can recover the virtual machine from its latest snapshot and keep running it elsewhere. If the hardware needs to be upgraded or changed, we can temporarily move the virtual machines to a different physical hosts.

Live migration is something that used to be unthinkable and impossible to achieve with physical hardware. In other words, thanks to virtualization, the deployment decisions you make can be easily changed also after the system has been started. As we will see, this can also help with elastic scalability, load balancing, or workload consolidation.

Inverted Hypervisor



The **hypervisor** supports multiple isolated virtual machines sharing the same hardware



The **inverted hypervisor** runs one virtual machine over multiple distributed hosts

What is the purpose of the hypervisor? The hypervisor multiplexes multiple virtual machines over one shared physical machine. This helps with using the hardware as efficiently as possible. The single physical processor will be mapped to multiple virtual processors that are going to run over it. If each application is not always fully utilizing its processor, it would be inefficient to allocate a full physical processor to it. By sharing the hardware, we can make sure it gets fully utilized all the time by multiple applications.

If we flip the structure then we have the inverted hypervisor. Here we can take one application and its operating system and then run it across multiple physical hardware resources. What if the hardware that you have is not powerful enough for one application? What if there it not enough memory? To reach the capacity needed to run your application, you can compose multiple pieces of hardware and thanks to the inverted hypervisor, they will look like it's a single virtual machine, a very large one, which your application can take advantage of. It is the inverted hypervisor responsability to aggregate multiple physical hardware resources and partitions the large application's workload among them.

Virtual Hardware = Software

Deployment Configuration and Resource Allocation (CPU, RAM, Disk, Network) becomes part of the Software code (and should be managed accordingly with versioning, change management, testing)

Different, independent and isolated deployment configurations should be used at different stages of the software production pipeline (never mix production and testing environments)

Virtual hardware becomes software. This is the second technological advance that brought us into the age of continuity. The first one was that the production of software is software too. Now we also realize that the hardware itself becomes software.

All of the configuration details about the container that you need in order to be able to run the software in production becomes a piece of software. This is specified using the corresponding language and then you can use version control, the proper change management processes. You can even do testing with it to make sure it doesn't get modified incorrectly.

We saw that in the software production pipeline we have to specify configuration for the integration stage, the capacity stage, the acceptance stage and the production stage. This configuration is not only the configuration of your software application, but it's also a configuration of the virtual hardware environment that you will use to run the system. This configuration may change as your software evolves (adding features requires more processing power). It may also change as a given version is running (adding more users requires more memory). Resource allocation becomes as easy as pushing a new commit of a configuration file. Let the Cloud provider scramble to install the hardware capacity that you allocated with a few keystrokes.

This configuration information becomes part of your software. The code you write to implement your component needs to include both the tests that will be run during the pipeline but also the specification of the virtual execution environment in which it will be deployed into.

This lecture on deployability concludes our transition from design time to run time. In the next lectures we will discuss how to design architectures which can deliver critical runtime qualities: scalability, availability and flexibility.

References

- Jez Humble, David Farley, Continuous Delivery. Reliable Software Releases Through Build, Test, and Deployment Automation Addison-Wesley, 2010, ISBN 978-0-321-60191-9
- Len Bass, Ingo Weber, Liming Zhu. DevOps: A Software Architect's Perspective, 2015, ISBN 978-0134049847
- Michael T. Nygard, Release It! Design and Deploy Production-Ready Software, 2018, ISBN 978-1-68050-239-8
- Gene Kim, Kevin Behr, George Spafford, The Phoenix Project: A Novel about IT, DevOps, and Helping Your Business Win, 2013, ISBN 978-0988262591
- Mendel Rosenblum, **The Reincarnation of Virtual Machines**, ACM Queue, Volume 2, issue 5, August 31, 2004
- **The Twelve-Factor App**
- **Open Container Initiative**
- XKCD 1319: **Automation**

Software Architecture

Scalability

10

Contents

- Scalability: Workloads and Resources
- Scale up or Scale out?
- Scaling Dimensions: Number of Clients (Workload), Input Size, State Size, Number of Dependencies
- Location Transparency: Directory and Dependency Injection
- Scalability Patterns: Scatter/Gather, Master/Worker, Load Balancing and Sharding